Fundamentos de Java Script

Desarrollo web Full Stack

Zaida Uricoechea

1. **Fundamentos de JavaScript**

Definición:

JavaScript es un lenguaje de programación o de secuencias de comandos que te permite implementar funciones complejas en páginas web; permite crear contenido que se actualiza dinámicamente, controlar multimedia, animar imágenes etc.
En una página web, HTML y CSS son estáticos; JavaScript se encarga de lo que **hace** el sitio:

- **Interactividad en el navegador:** Crear menús desplegables, galerías de imágenes (carruseles), ventanas emergentes, mapas interactivos y animaciones.
- **Validación de formularios:** Revisar que los datos escritos por un usuario estén correctos antes de enviarlos al servidor.
- **Actualización en tiempo real:** Mostrar nuevo contenido (como mensajes en redes sociales o comentarios) sin necesidad de recargar la página completa.
- **Desarrollo del lado del servidor (Backend):** Gracias a entornos como **Node.js**, JavaScript también se puede ejecutar fuera del navegador para construir servidores y bases de datos.

2. **Operadores de comparación:**

Los operadores de comparación son símbolos que se utilizan en programación y hojas de cálculo para evaluar la relación entre dos valores, devolviendo un resultado lógico de tipo verdadero (true / 1) o falso (false / 0).

Operador	Descripción	Uso	Resultado
===	Estrictamente igual	2 === 2	verdadero
!==	No es estrictamente igual	2 !== "2"	verdadero
==	Igual (o "aproximadamente igual")	2 == "2"	verdadero
!=	No igual	2 != "3"	verdadero
>	Mayor que	3 > 2	verdadero
>=	Mayor o igual que	2 >= 2	verdadero
<	Menor que	2 < 3	verdadero
<=	Menor o igual que	2 <= 3	verdadero

Coerción e igualdad de tipos

Dos de los operadores de comparación más usados son **==** para la igualdad flexible y **===** para la igualdad estricta. **==** realiza una comparación imprecisa entre dos valores forzando a los operandos a que coincidan con los tipos de datos, si es posible. Por ejemplo, 2 == "2" muestra true, aunque la comparación se realice entre un valor numérico y un valor de cadena.

```
2 == 2
> true

2 == "2"
> true
```

Lo mismo sucede con **!=**, que muestra true solo si los operandos que se comparan *no* son iguales de forma no estricta.

```
2 != 3

> true

2 != "2"

> false
```

Las comparaciones estrictas con === o !== no realizan una coerción de tipo. Para que una comparación estricta evalúe a true, los valores que se comparan deben tener el mismo tipo de datos. Debido a esto, 2 == "2" muestra true, pero 2 === "2" muestra false:

```
2 === 3

> false

2 === "2"

> false
```

Los operadores de igualdad (== y !=) Utilizan el algoritmo de comparación de igualdad abstracta para comparar dos operandos. Esto se puede resumir a grandes rasgos como:

- Si los operandos ambos son objetos, devuelve truesolo si ambos operandos hacen referencia al mismo objeto.
- Si un operando es null y el otro undefined, devuelve verdadero(true).
- Si los operandos son de diferente tipos, intenta convertirlos al mismo tipo antes de comparar:
    - Al comparar un número con una cadena, convierte la cadena en un valor numérico.
    - Si uno de los operandos es booleano, convierte el operando booleano en 1 si es verdadero y en 0 en el caso de falso.
    - Si uno de los operandos es un objeto y el otro es un número o una cadena, convierte el objeto en una primitiva utilizando los métodos valueOf() y toString() del objeto.
- Si los operandos tienen el mismo tipo, se comparan de la siguiente manera:
    - String: devuelve verdadero solo si ambos operandos tienen los mismos caracteres y en el mismo orden.
    - Number: devuelve verdadero solo si ambos operandos tienen el mismo valor. +0 y -0 se tratan como el mismo valor. Si alguno de los operandos es NaN, devuelve falso.
    - Boolean: retorna verdadero solo si ambos operandos son verdaderos o falsos.

La diferencia más notable entre este operador y el operador de igualdad estricta (===) es que el operador de igualdad estricta no realiza la conversión de tipos.

Ejemplos:

```
"1" == 1; // true
1 == "1"; // true
0 == false; // true
0 == null; // false
0 == undefined; // false
0 == !!null; // true, Operador Logico NOT
0 == !!undefined; // true, Operador Logico NOT
null == undefined; // true

const number1 = new Number(3);
const number2 = new Number(3);
number1 == 3; // true
number1 == number2; // false
```

```
const object1 = { key: "value" };
const object2 = { key: "value" };

object1 == object2; // false
object2 == object2; // true
```

3. **Operadores Logicos:**

Un operador lógico es uno o dos símbolos o una palabra clave que le indica a la computadora cómo combinar sentencias condicionales. El resultado de usar un operador lógico es un valor booleano (`true`o `false`).

Los operadores lógicos más comunes son:

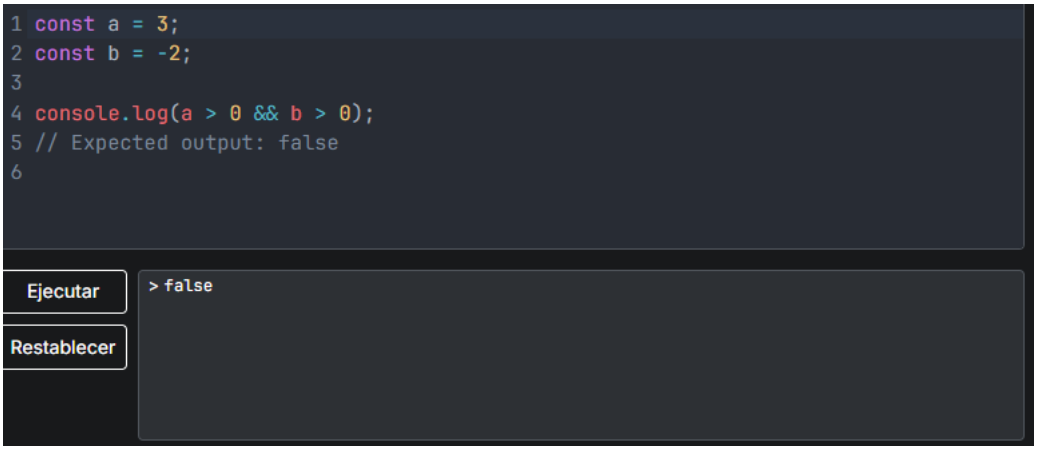
- &&(AND lógico)
- ||(OR lógico)
- !(NOTA lógica)

Operador lógico AND (&&)

El operador lógico AND (&&) (conjunción lógica) será true para un conjunto de operandos booleanos si y solo si todos los operandos son true. En caso contrario, será false.

Generalmente, el operador retorna el valor del primer operando *falsy* encontrado cuando evalúa de izquierda a derecha, o el valor del último operando si todos ellos son *truthy*.

El **operador &&** devuelve truesi **ambas expresiones** son trueverdaderas, de lo contrario false:




```
a1 = true && true; // t && t retorna `true`
a2 = true && false; // t && f retorna `false`
a3 = false && true; // f && t retorna `false`
a4 = false && 3 === 4; // f && f retorna `false`
a5 = "Cat" && "Dog"; // t && t retorna "Dog"
a6 = false && "Cat"; // f && t retorna `false`
a7 = "Cat" && false; // t && f retorna `false`
a8 = "" && false; // f && f retorna ""
a9 = false && ""; // f && f retorna `false`
```

Aunque el operador && puede ser usado con operandos que no son valores booleanos, aún puede ser considerado un operador booleano ya que su valor de

retorno siempre puede ser convertido a un booleano primitivo. Para explícitamente convertir su valor de retorno (o cualquier expresión en general) al correspondiente valor booleano, se debe usar un doble (operador lógico NOT) o el contructor del objeto Boolean.

Operador lógico OR (||)

El **operador lógico OR (||)** (disyunción lógica) es verdadero para un conjunto de operandos si y solo si uno o más de sus operandos es verdadero. Es normalmente usado con valores booleanos (lógicos). Cuando es así, retorna un valor booleano. Sin embargo, el operador || puede retornar el valor de uno de los operandos especificados, por lo que si este operador es usado con valores no booleanos, retornará un valor no booleano.

l **operador ||** devuelve true si **una o ambas expresiones** son true verdaderas, de lo contrario false:

```
true || true; // t || t retorna `true`
false || true; // f || t retorna `true`
true || false; // t || f retorna `true`
false || 3 === 4; // f || f retorna `false`
"Cat" || "Dog"; // t || t retorna "Cat"
false || "Cat"; // f || t retorna "Cat"
"Cat" || false; // t || f retorna "Cat"
"" || false; // f || f retorna `false`
false || ""; // f || f retorna ""
false || varObject; // f || objeto retorna varObject
```

Operador lógico NOT:

Una expresión lógica NOT (!) niega el valor verdadero o falso de un operando, y se evalúa como true si el operando se evalúa como false, y como false si el operando se evalúa como true. El operador NOT (!) devuelve true para **expresiones falsas** y false para **expresiones verdaderas**.

- El operador NOT sólo afecta a la expresión que sigue inmediatamente, a menos que una expresión lógica más compleja esté entre paréntesis.
- Puede sustituir ~ o ¬ por NOT como operador lógico.
- NOT se puede utilizar para comprobar si una variable numérica tiene el valor 0, 1 o cualquier otro valor. Por ejemplo, todas las variables reutilizables se inicializan en 0. Por lo tanto, NOT (#ID) devuelve false o falta cuando a *#ID* se le ha asignado un valor distinto de 0.

```
let usuarioAutenticado = false;
let accesoDenegado = !usuarioAutenticado; // El resultado será true
```

```
let edad = 15;
if (!(edad >= 18)) {
  // Entrará aquí porque (edad >= 18) es false, y el NOT (!) lo invierte a tr
  console.log("Acceso restringido para menores de edad.");
}
```

4. **Condicionales:**

Declaraciones condicionales

Las sentencias condicionales nos permiten realizar **diferentes acciones** para **diferentes condiciones** .

Las sentencias condicionales **ejecutan un código diferente** dependiendo de si las condiciones **son verdaderas** o **falsas** .

Las sentencias condicionales incluyen:

- si
- si...si no
- si...si no si...si no

- cambiar
- ternario (? :)

Tipos de condicionales:

- Se utiliza if para especificar un bloque de código que se ejecutará si se cumple una condición específica. true
- Se utiliza else para especificar un bloque de código que se ejecutará si se cumple la misma condición. false
- Se utiliza else if para especificar una nueva condición a probar, si la primera condición es false
- Se utiliza switch para especificar varios bloques de código alternativos que se ejecutarán.
- Utilice (? :)(ternario) como abreviatura de if...else

❖ Condicional if

Con él podemos indicar en el programa que se tome un camino sólo si se cumple la **condición** que establezcamos.

Ejemplo:

```
let nota = 7;
console.log("He realizado mi examen.");

// Condición (si la nota es mayor o igual a 5)
if (nota ⩾ 5) {
  console.log("¡Estoy aprobado!");
}
```

En este caso, como el valor de nota es superior o igual a 5, nos aparecerá en la consola el mensaje «¡Estoy aprobado!». Sin embargo, si modificamos en la primera línea el valor de nota a un valor inferior a 5, no nos aparecerá ese mensaje.

```
let edad = 20;

if (edad >= 18) {
  console.log("Eres mayor de edad");
}
```

En este caso como la edad es “20” nos aparecerá en la consola el mensaje “eres mayor de edad”, pero si tuviese un número menor a 18 no aparecería nada.

- Condicional else

Se utiliza else para especificar un **bloque de código** que se ejecutará si se cumple la misma condición false.

Con if se puede ejecutar un conjunto de instrucciones solo si la condición es verdadera. Si queremos que se ejecute *otro* conjunto de instrucciones cuando la condición es falsa, entonces podemos usar una **cláusula "else"**.

Ejemplos:

```
let password = "TOPSECRET";

if (password === "TOPSECRET") {
    console.log("¡Acceso concedido!"); // Se ejecuta si es verdadero
} else {
    console.log("Contraseña incorrecta, intenta de nuevo."); // Se ejecuta si es
}
```

- Condicional else if:

Se puede dar el caso que queramos establecer una **alternativa** a una condición. Para eso utilizamos el if seguido de un else. Con esto podemos establecer una acción A si se cumple la condición, y una acción B si no se cumple.

Ejemplo:

```
let nota = 7;
console.log("He realizado mi examen. Mi resultado es el siguiente:");

if (nota < 5) {
  // Acción A: nota es menor que 5
  calificacion = "suspendido";
} else {
  // Acción B: Cualquier otro caso diferente a A (nota es mayor o igual que 5)
  calificacion = "aprobado";
}

console.log("Estoy", calificacion);
```

En este ejemplo comprobaremos que podemos conseguir que se muestre el mensaje **Estoy aprobado** o **Estoy suspendido** dependiendo del valor que tenga la variable nota.

```
let edad = 16;

if (edad >= 18) {
  console.log("Puedes votar");
} else {
  console.log("No puedes votar");
}
```

En este ejemplo sucede lo mismo, muestra un ejemplo de que si es menor de 18, aparece el mensaje “no puedes votar”

5. **Arrays**

Un Array es un tipo de objeto diseñado para almacenar colecciones de datos.

Las características clave de los arrays de JavaScript son:

- Elementos : Un array es una lista de valores, conocidos como elementos.
- Ordenado : Los elementos de la matriz se ordenan según su índice.

- Índice cero : El primer elemento está en el índice 0, el segundo en el índice 1, y así sucesivamente.
- Tamaño dinámico : Los arreglos pueden crecer o encogerse a medida que se agregan o eliminan elementos.
- Heterogéneos : Los arreglos pueden almacenar elementos de diferentes tipos de datos (números, cadenas, objetos y otros arreglos).

En algunas ocasiones también se les suelen llamar **arreglos** o **vectores**. En Javascript, se pueden definir de varias formas:

new Array(size) Crea un array vacío de tamaño size. Sus valores no están definidos, pero son .

new Array(e1, e2...) Crea un array con los elementos indicados.

[e1, e2...] Simplemente, los elementos dentro de corchetes: []. Notación preferida.

Si tienes una lista de elementos (una lista de nombres de autos, por ejemplo), almacenar los nombres en variables individuales podría verse así:

```
let car1 = "Saab";
let car2 = "Volvo";
let car3 = "BMW";
```

Sin embargo, ¿qué ocurre si quieres recorrer todos los autos y encontrar uno específico? ¿Y si no tuvieras 3 autos, sino 300?

¡La solución es un array!

Un array puede contener muchos valores bajo un mismo nombre, y se puede acceder a ellos haciendo referencia a un número de índice.

Creación de un array

Utilizar un literal de matriz es la forma más sencilla de crear una matriz en JavaScript.

Sintaxis:

const *array_name* = [*item1*, *item2*, ...];

Es práctica común declarar arreglos con la palabra clave const .

Ejemplo

const cars = ["Saab", "Volvo", "BMW"];

Los espacios y los saltos de línea no son importantes. Una declaración puede abarcar varias líneas:}

Ejemplo

```
const cars = [
 "Saab",
 "Volvo",
 "BMW"
];
```

También puedes crear un array vacío y añadir los elementos más adelante:

Ejemplo

```
const cars = [];
cars[0]= "Saab";
cars[1]= "Volvo";
cars[2]= "BMW";
```

Al contrario que muchos otros lenguajes de programación, Javascript permite que se puedan realizar arrays de **tipo mixto**, no siendo obligatorio que todos los elementos sean del mismo tipo de dato.

Alternativas para crear arrays

Existen otras formas que permiten generar un Array partiendo de otras estructuras de datos.

Algunos de estos casos son los siguientes:

Método	Descripción
Array.from(obj)	Intenta convertir el obj en un array.
Array.from(obj, fmap)	Idem, pero ejecuta la función fmap por cada elemento. Equivalente a .map()
Array.from({ length:size})	Crea un array a partir de un de tamaño size, relleno de UNDEFINED
.concat(e1, e2, e3...)	Devuelve los elementos pasados por parámetro concatenados al final del array.
.join(sep)	Une los elementos del array mediante separadores sep en un string.

Acceso a los elementos de un array

Al igual que los , saber el número elementos que tiene un array es muy sencillo. Sólo hay que acceder a la propiedad .length, que nos devolverá el número de elementos existentes en un array:

.length Propiedad que devuelve el número de elementos del array.

[pos] Operador que devuelve (o modifica) el elemento número pos del array.

.at(pos) Método que devuelve el elemento en la posición pos. Números negativos en orden inverso.

.with(pos, el) Método que crea una copia y modifica el elemento de la posición pos con el.

6. **Metodos básicos:**

Métodos básicos para agregar y eliminar elementos

- **push()**: Añade uno o más elementos al **final** del array y devuelve la nueva longitud.

  Este método es útil cuando se necesita agregar elementos a un array de forma dinámica, como agregar nuevos productos a un carrito de compras, registrar eventos de usuario, etc.

- **pop()**: Elimina el **último** elemento del array y lo devuelve.

  Este método es útil en varias aplicaciones, como en la implementación de pilas (stack), donde el último elemento agregado es el primero en ser eliminado (LIFO - Last In, First Out). También se puede utilizar en el manejo de listas donde se necesita procesar y eliminar elementos del final, como en una lista de tareas donde se eliminan las más recientes primero, o en la manipulación de datos donde se requieren acciones secuenciales desde el último al primer elemento. Además, pop() es útil en la reducción gradual de elementos en algoritmos de procesamiento que necesitan iterar desde el final de un array.

- **unshift()**: Añade uno o más elementos al **inicio** del array.}

  Esta funcionalidad es útil en diversas aplicaciones prácticas, como el manejo de listas o colas donde los elementos más nuevos deben ser procesados primero, en el seguimiento de historiales de navegación web para mantener las páginas más recientes al inicio, o en la implementación de estructuras de datos como colas doblemente terminadas (deques). También es útil en escenarios donde se necesita mantener un orden específico de llegada, como en el trazado de rutas o la gestión de flujos de trabajo.

- **shift()**: Elimina el **primer** elemento del array y lo devuelve.

  Es útil en diversas aplicaciones prácticas, como en la implementación de colas (FIFO - First In, First Out), donde el primer elemento agregado es el primero en ser eliminado. También se emplea en el manejo de listas de espera, donde se procesa y elimina el primer elemento en la fila. Además, shift() es útil en algoritmos que requieren iterar y eliminar elementos desde el inicio de un

array, como en la manipulación de datos o en tareas de procesamiento por lotes.

EJEMPLO:

Imagina que push y pop trabajan con el **final** de la fila, mientras que unshift y shift trabajan con el **inicio**.

```
let frutas = ['🍎 Manzana', '🍌 Plátano'];

// push(): Agrega al FINAL
frutas.push('🥝 Kiwi');
// Resultado: ['🍎 Manzana', '🍌 Plátano', '🥝 Kiwi']

// pop(): Elimina el ÚLTIMO
frutas.pop();
// Resultado: ['🍎 Manzana', '🍌 Plátano']

// unshift(): Agrega al INICIO
frutas.unshift('🍓 Fresa');
// Resultado: ['🍓 Fresa', '🍎 Manzana', '🍌 Plátano']

// shift(): Elimina el PRIMERO
frutas.shift();
// Resultado: ['🍎 Manzana', '🍌 Plátano']
```

Métodos básicos de búsqueda y verificación

- **includes()**: Comprueba si el array contiene un elemento determinado, devolviendo true o false.

  Es útil en aplicaciones que requieren verificar la presencia de un valor dentro de una lista, como en la validación de entradas de usuario, la búsqueda de elementos específicos en colecciones de datos, o la implementación de lógica condicional basada en la existencia de ciertos elementos. También se emplea en la gestión de estados y en la interacción con datos dinámicos donde es crucial detectar la presencia o ausencia de elementos.

- **indexOf() y lastIndexOf**(): Retorna la primera/última posición en la que se encuentra un elemento en el array, respectivamente. Ambos métodos devuelven el índice del elemento encontrado o -1 si el elemento no está presente.Array.

Son útiles en aplicaciones que requieren la búsqueda de elementos específicos en una lista, como la validación de existencia de datos, la gestión de selecciones de usuarios, o la manipulación de datos para operaciones de actualización o eliminación. También se emplean en la implementación de algoritmos de búsqueda y en la interacción con colecciones de datos dinámicas.

- isArray(): Método estático que determina si el valor pasado es un array. [
- find(): Retorna el primer elemento que cumple con la condición especificada en una función de prueba.

  Es útil en aplicaciones que requieren la búsqueda de un elemento específico dentro de una lista, como encontrar un objeto con una propiedad particular, localizar el primer valor que cumple una condición, o buscar elementos en base a criterios dinámicos. También se emplea para implementar funcionalidades de búsqueda y filtrado en interfaces de usuario y sistemas de gestión de datos.

- **filter():** Crea **un nuevo array con todos los elementos que cumplan con la condición** especificada en una función de prueba.

  Es útil en aplicaciones que requieren la extracción de subconjuntos de datos que cumplen con ciertos criterios, como filtrar productos por precio, seleccionar elementos activos en una lista, o buscar registros que cumplan con condiciones específicas. También se emplea en la limpieza de datos, la implementación de funcionalidades de búsqueda avanzada y la creación de vistas filtradas en interfaces de usuario.

  EJEMPLO:

```
let alumnos = ['Ana', 'Luis', 'Pedro'];

// includes(): ¿Está en la lista? (Devuelve true o false)
let tieneLuis = alumnos.includes('Luis'); // true
let tieneJuan = alumnos.includes('Juan'); // false

// indexOf(): ¿En qué posición está? (Recuerda que se cuenta desde 0)
let posicionAna = alumnos.indexOf('Ana');   // 0
let posicionPedro = alumnos.indexOf('Pedro'); // 2
let posicionJuan = alumnos.indexOf('Juan');   // -1 (Significa que no existe)
```

Métodos básicos de transformación e iteración

- **slice()**: Devuelve una copia superficial (shallow copy) de una porción del array en un nuevo array (no modifica el original).

  Es útil en diversas aplicaciones prácticas, como cuando se necesita trabajar con una sublista de elementos sin alterar la lista original, en la paginación de resultados donde se extraen porciones específicas de datos, o al clonar arrays para evitar efectos secundarios al modificar datos. También se emplea en la manipulación de listas donde se requiere obtener segmentos de datos para su procesamiento o análisis.

- **splice()**: Cambia el contenido de un array eliminando, reemplazando o agregando elementos (modifica el array original).

  Es extremadamente versátil y se puede emplear en una variedad de aplicaciones prácticas, como en la modificación de listas donde se necesita insertar elementos en una posición específica, eliminar elementos de cualquier parte del array o reemplazar elementos existentes con nuevos valores. También es útil en algoritmos de procesamiento de datos que requieren manipulaciones complejas del array, como en la gestión de inventarios, la edición de secuencias de operaciones, o cualquier situación en la que se necesiten modificaciones controladas del contenido del array.

- **concat()**: Combina dos o más arrays en uno nuevo.

  Es útil en aplicaciones donde se necesita combinar múltiples listas de elementos en una sola, como en la fusión de datos de diferentes fuentes, la concatenación de resultados paginados, o la construcción de listas compuestas de elementos relacionados. También se emplea en situaciones donde se requiere agregar elementos adicionales a un array existente sin modificar el array original.

- **join()**: Une todos los elementos de un array en una cadena de texto (string).

- **forEach()**: Ejecuta una función indicada una vez por cada elemento del array.

  Es útil en una variedad de aplicaciones, como en el procesamiento de datos, donde se necesita realizar operaciones en cada elemento de una lista, en la generación de informes o visualizaciones, y en la manipulación de arrays sin crear nuevos arrays. También se emplea para recorrer listas y aplicar transformaciones, realizar cálculos acumulativos o interactuar con cada elemento para tareas de logging, validación o actualización de UI.

- **map()**: Crea un nuevo array con los resultados de la llamada a la función aplicada por cada elemento.

  Es útil en aplicaciones que requieren la transformación de datos, como cambiar el formato de datos, realizar cálculos o extracciones, y generar nuevas estructuras de datos basadas en las existentes. También se emplea en situaciones donde se necesita un array modificado sin alterar el original, como en la creación de listas de elementos renderizables en interfaces de usuario, o en la manipulación de datos para análisis y procesamiento posterior.

- **filter()**: Crea un nuevo array con todos los elementos que cumplan la condición implementada por la función.

  EJEMPLO:

```
// concat(): Une dos listas en una nueva
let hombres = ['Luis', 'Pedro'];
let mujeres = ['Ana', 'María'];
let todos = hombres.concat(mujeres);
// Resultado: ['Luis', 'Pedro', 'Ana', 'María']

// join(): Une todo en un solo texto separado por lo que elijas
let ingredientes = ['Pan', 'Queso', 'Tomate'];
let receta = ingredientes.join(' + ');
// Resultado: "Pan + Queso + Tomate"
```

7. Ciclos

Un ciclo (también llamado bucle o *loop*) es una estructura de control en programación que permite repetir un bloque de código varias veces de forma automática, sin necesidad de escribir las mismas líneas una y otra vez.

Los ciclos son fundamentales para evitar la redundancia y procesar grandes volúmenes de datos de manera eficiente.

Componentes de un ciclo

Para que un ciclo funcione correctamente y no se repita de forma infinita, por lo general necesita tres elementos esenciales:

- **Inicio (o variable de control):** El punto de partida o el estado inicial en el que comienza el ciclo.

- **Condición de permanencia:** Una prueba lógica que el programa evalúa antes (o después) de cada repetición. Si la condición es **verdadera**, el ciclo continúa; si es **falsa**, el ciclo se detiene.
- **Actualización (o incremento):** El cambio que se realiza en cada vuelta para acercar el ciclo a su final (por ejemplo, sumar 1 a un contador).

FOR:

un ciclo for se repite hasta que una condición especificada se evalúe como false. El bucle for de JavaScript es similar al bucle for de Java y C.

```
for ([initializacion]); [condicion]; [expresion-final]) {
   // sentencias
}
```

Consiste de tres expresiones y una declaración:

Descripción

- inicialización - Sucede antes de la primera ejecución del bucle. Esta expresión es comúnmente utilizada para crear contadores. Las variables creadas tienen un alcance (scope) limitado al cuerpo del bucle. Una vez que el bucle ha terminado su ejecución las variables son destruidas.
- condición - Expresión que es evaluada antes de la ejecución de cada iteración. Si se omite, esta expresión es evaluada como verdadera. Si devuelve true, la sentencia del cuerpo del bucle se ejecuta. Si devuelve false, el bucle se detiene.
- expresión-final - Expresión que se ejecuta luego de cada iteración. Usualmente es utilizada para incrementar un contador. Pero también puede ser utilizada para decrementar el contador.
- sentencia o declaración - Código que será ejecutado repetidamente por el bucle

Cuando se ejecuta un bucle for, ocurre lo siguiente:

1. Se ejecuta la expresión de iniciación *expresiónInicial*, si existe. Esta expresión normalmente inicia uno o más contadores de bucle, pero la sintaxis permite una expresión de cualquier grado de complejidad. Esta expresión también puede declarar variables.
2. Se evalúa la expresión expresiónCondicional. Si el valor de expresiónCondicional es verdadero, se ejecutan las instrucciones del bucle. Si el valor de condición es falso, el bucle for termina. (Si la expresión condición se omite por completo, se supone que la condición es verdadera).
3. Se ejecuta la instrucción. Para ejecutar varias instrucciones, usa una declaración de bloque ({ ... }) para agrupar esas declaraciones.
4. Si está presente, se ejecuta la expresión de actualización expresiónDeActualización.
5. El control regresa al paso 2.

EJEMPLO:

Usa for cuando sabes exactamente cuántas veces quieres hacer algo. En este caso, sabemos que queremos contar **exactamente 3 veces**.

```
for (let i = 1; i <= 3; i++) {
  console.log("Número " + i);
}
```

Resultado en pantalla:

- Número 1
- Número 2
- Número 3

WHILE:

Una declaración while ejecuta sus instrucciones siempre que una condición especificada se evalúe como true.

Si la condición se vuelve false, la instrucción dentro del bucle se deja de ejecutar y el control pasa a la instrucción que sigue al bucle.

```
while (condicion)

{

  sentencia(s);

}
```

sentencia(s): Una sentencia es código que se ejecuta si la condición devuelve verdadero (true).

condición: *Es una expresión booleana (Boolean) que es evaluada antes de cada paso (iteración) por el bucle. Si esta condición evalúa a verdadero* (true), las sentencias son ejecutadas. Cuando la condición evalúa a falso (false), la ejecución continúa con la siguiente sentencia luego del bucle while.

EJEMPLO:

Usa while cuando dependes de una condición que cambia. El ciclo se repite **mientras la batería tenga energía**. Cuando llega a 0, se detiene.

```
let bateria = 3;

while (bateria > 0) {
  console.log("El teléfono está encendido. Batería: " + bateria);
  bateria = bateria - 1; // Le restamos 1 en cada vuelta
}

console.log("El teléfono se apagó.");
```

Resultado en pantalla:

- El teléfono está encendido. Batería: 3
- El teléfono está encendido. Batería: 2
- El teléfono está encendido. Batería: 1
- El teléfono se apagó.